

Automation of Apache Spark Deployment on Google Cloud Platform

Infrastructure as Code Configuration Management Report

MEKKAOUI Ossama Moussa
HASSAIN Ayoub

January 22, 2026

Abstract

This report details the design, implementation, and automation of a distributed Apache Spark cluster on Google Cloud Platform (GCP). Starting from a basic infrastructure, the project evolved into a secure, production-grade architecture featuring network isolation, an Edge node for job submission, and centralized NFS storage. The deployment pipeline leverages Terraform for infrastructure provisioning and Ansible for configuration management, demonstrating a full DevOps lifecycle.

1 Introduction

The objective of this project was to automate the deployment of a Big Data environment without manual intervention. Our goal was to build a reproducible, scalable system. We utilized **Terraform** to manage GCP resources (VMs, VPCs, Firewalls) and **Ansible** to configure software (Java, Spark, NFS) across the cluster.

2 Infrastructure Evolution

2.1 Phase 1: Initial Prototype

Our initial architecture consisted of three simple Compute Engine instances: one Master and two Workers within a custom VPC.

- **Challenge:** All nodes had public IPs and required manual updates to the Ansible inventory whenever Terraform was re-applied.
- **Limitation:** Security was minimal, with the Master node acting as both controller and job submitter, exposing critical ports to the internet.

2.2 Phase 2: Security The Edge Node Strategy

To adhere to industry best practices, we refactored the architecture to isolate the compute resources.

- **The Edge Node:** We introduced a specialized "Edge" (or Jump) node. This is the *only* machine accessible via SSH from the outside world.
- **Network Isolation:** The Master and Worker nodes now reside in a private subnet. They have no external IP addresses and can only be reached through the Edge node.

- **Implementation:** We used Terraform to output the Edge node's public IP and configured Ansible to use it as a `ProxyCommand`. This ensures that our automated playbooks can tunnel through the Edge node to configure the private workers seamlessly.

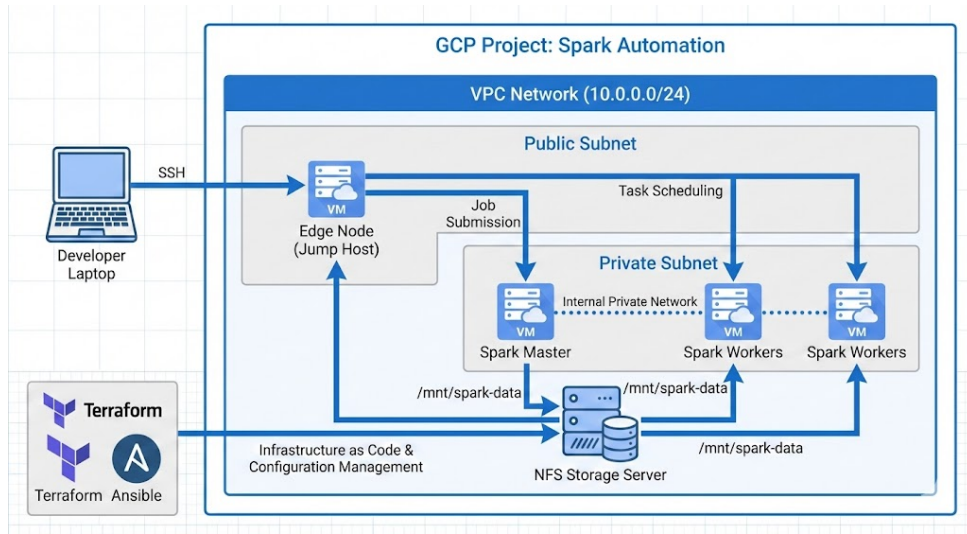


Figure 1: Final Architecture: Edge Node acting as the gateway to the private cluster.

3 Automation Pipeline

3.1 Terraform: Dynamic Inventory Generation

A major hurdle was the dynamic nature of cloud IPs. Hardcoding IP addresses in Ansible is fragile. We solved this by using Terraform's `local_file` resource.

- Terraform provisions the infrastructure.
- It captures the internal IPs of the Master/Workers and the external IP of the Edge node.
- It automatically generates the `hosts.ini` file required by Ansible.

```

1 resource "local_file" "ansible_inventory" {
2   content = templatefile("inventory.tmpl", {
3     edge_ip = google_compute_instance.edge.network_interface.0.access_config.0.
      nat_ip
4     master_ip = google_compute_instance.master.network_interface.0.network_ip
5   })
6   filename = "../ansible/inventory/hosts.ini"
7 }

```

Listing 1: Terraform Output Logic

3.2 Ansible: Configuration Management

We structured our Ansible code into modular **Roles**:

- **Java Role:** Installs OpenJDK 11 on all nodes.
- **Spark Role:** Downloads Spark 3.5.0, configures `spark-env.sh`, and sets up Systemd services for auto-restart resilience.
- **NFS Role:** Configures the Storage node as a server and the Cluster nodes as clients.

4 Data Management: The Storage Node

Initially, we faced a data redundancy issue where benchmark datasets had to be downloaded to every worker individually. To solve this, we implemented a centralized storage solution.

4.1 Why NFS?

We deployed a dedicated **NFS (Network File System)** server.

- The NFS server exports a `/data` directory.
- All Workers and the Edge node mount this directory at `/mnt/spark-data`.
- **Benefit:** We download datasets once to the storage node, and they are instantly available to the entire cluster. This simulates a decoupled storage/compute architecture common in enterprise environments.

5 Validation Benchmarking

5.1 Java Migration

While initial tests were written in Python (PySpark), we migrated our final benchmark application to **Java**. This required updating our Ansible pipeline to install Maven on the Edge node, compile the source code into a JAR file, and submit it to the cluster. This aligns with the native JVM architecture of Spark.

5.2 WordCount Results Performance Analysis

We validated the cluster using a "WordCount" algorithm on a dataset of **10 classic novels** (Project Gutenberg edition). The text files were read via the shared NFS storage using the `file://` protocol.

We conducted performance benchmarks with varying executor configurations (see Figure 2):

- **1 Executor:** The baseline execution time was approximately **28 seconds**.
- **2 Executors:** Performance improved significantly, dropping to **19 seconds**. This confirms that the workload was successfully distributed.
- **4 Executors:** Surprisingly, increasing to 4 executors yielded **no additional speedup** (remaining at 19 seconds).

Analysis: The lack of improvement beyond 2 workers suggests we hit a bottleneck unrelated to compute power—likely the network latency of the NFS storage or the small size of the dataset (6MB), where the overhead of scheduling tasks outweighs the benefits of parallelization.

The successful generation of part-files in the shared directory confirmed that the Workers could communicate, access shared data, and process tasks in parallel.

6 Conclusion

This project successfully demonstrated the power of Infrastructure as Code. We evolved from a simple manual setup to a fully automated, secure platform. Key achievements include:

1. **Zero-Touch Deployment:** One command (`terraform apply`) builds the infrastructure; and ansible commands do the configurations (`ansible-playbook`).

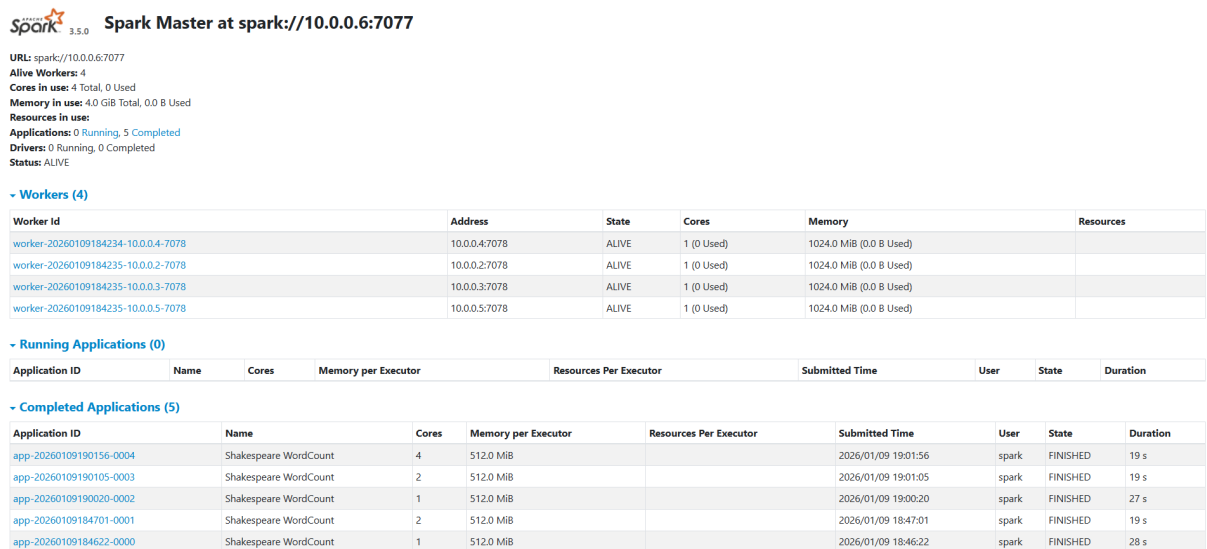


Figure 2: Spark Web UI showing execution times: 28s (1 core), 19s (2 cores), and 19s (4 cores).

2. **Security:** Implementation of a Jump Host (Edge Node) architecture.
3. **Persistence:** Integration of NFS for shared cluster storage.

Future improvements would include implementing HDFS for larger datasets and adding auto-scaling policies based on CPU load.